APPLICATION OF MATRIX MULTIPLICATION TO ALGORITHM COMPLEXITY

Performance Comparison of Python, Java and C

Individual Assignment 01 Experimental Report

Rubén Perdomo Aspas

Repositorio: https://github.com/RubenPerdomo159/BigData-Individual-Assigments.git

1. INTRODUCTION.

In scientific computing and data analysis had a common operation, the Matrix multiplication, a useful example for studying the complexity of an algorithm because the number of operations increases quickly as the matrix become larger. This Assigment 01 aims to implements the same basic matrix multiplication in Python, Java and C, in order to compare their execution times using the same matrix sizes.

The experiments focuses on the multiplication kernel itself rather tan the time needed to start a program, read files or write results. The results show how programming language and implementations details can affect the execution time on algorithm with the same theoretical complexity

2. ALGORITHM AND COMPLEXITY.

The programs takes two square matrices, A and B with n rows an n columns each other, to produce a result matrix C. It uses three nested loops, one outer takes a row and a column, the inner one calculates the dot product that we will need for one element of the result.

$$C[i][j] = \sum A[i][k] * B[k][j], \text{ fo } k = 0, \ldots, n-1$$

For every n^2 element of the result, the algorithm will perform almost n multiplications and n additions. As a result, its time complexity is $O(n^3)$. If n is multiplied by 2, the main amount of work is expected to increase by a factor of 8, always if the conditions are similar.

All programs generate deterministic matrix values by using the same formula and sizes. Each size have a warm-up followed by five measured repetitions.

3. CORRECTNESS AND MEASUREMENT METHOD.

Before all the main process, each implementation does a run correctness checks. These include a small matrix with an anwser already known, multiplications that involves an identity matrix and a zero matrix, and a comparasion with another multiplication order as a reference. The raw CSV report correct=true for every recorded run.

Execution time is measured in nanoseconds around the multiplication kernel. The report uses the median of the five measured runs for the comparasion, it would be useful, because is less afected tan the mean on a single unexpected high or low measurement. The CSV summaries contains quartiles and interquartiles range, which indicates the variation between repetitions.

The memory column estimates the elements storage of the three matrices as $3 * n^2 * 8$ bytes, if each element occupies 8 bytes. It isn't a total process memory measurement. Python lists and objects can require additional memory beyond the raw element payload.

4. EXPERIMENTAL ENVIRONMENT.

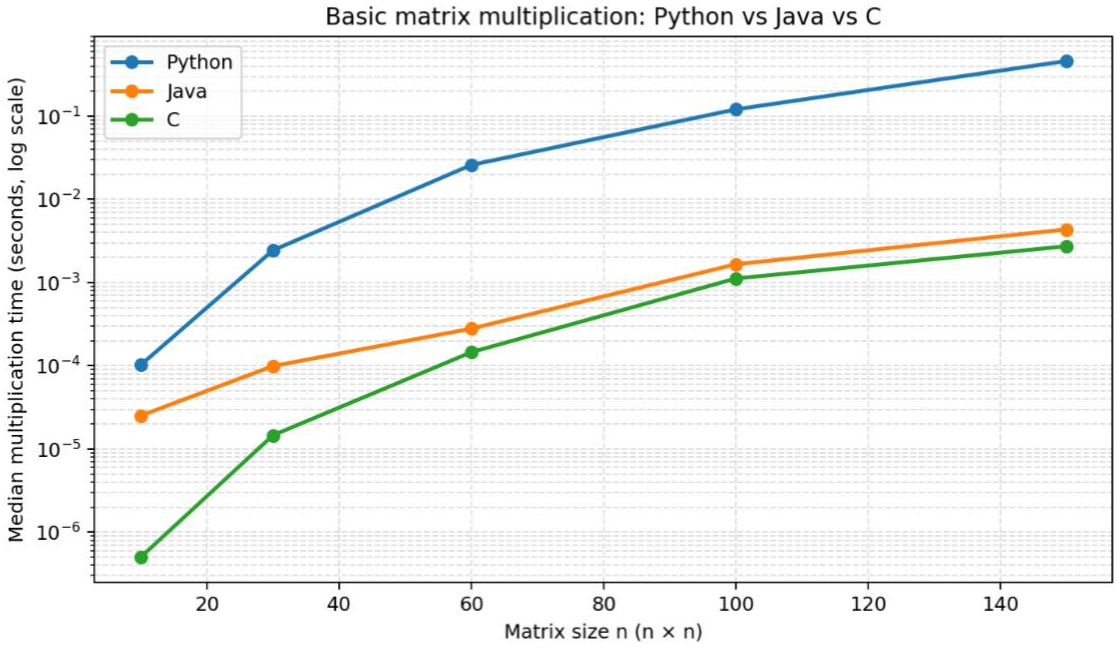
Item	Recorded information
Operating system	Windows 11
Architecture	64-bit x86
Python	3.13.12
Java	26.0.1, Java HotSpot 64-Bit Server VM
C compiler	MSYS2
Logical CPUs	8

Each run was recorded at a different time, Python and Java identifies the operating system and architecture, on the other hand, C environment file still contains placeholders for the processor and compilers. The comparision should be treated as an experiment on the availiable setup rather than a fully controlled benchmark with every hardware and compiler detail documented.

5. RESULTS.

The following table shows the median multiplication time for each size, units are converted to make the values easier to read.

Matrix size	Python median	Java median	C median
10	0.0102 ms	25.10 µs	0.50 µs
30	2.442 ms	99.20 µs	14.60 µs
60	25.943 ms	0.279 ms	0.146 ms
100	0.121 s	1.659 ms	1.120 ms
150	0.461 s	4.359 ms	2.732 ms



6. DISCUSSION.

6.1. Effect of matrix size

The execution time increases with matrix size in all three implementations, this agrees with the $O(n^3)$ complexity of the algorithm expected. In Python, for example, the median rises from about 0.000102 at n=10 to 0.461s at n=150, this increase is substantial because the number of operations grows in three dimensions.

6.2. Comparision between languages

At n=150, the median time reaches 0.461s in Python, while in Java is 4.359ms, and on C is 2.732ms. In this case, Python takes more time, almost 170 times slower than C, and Java 1.6 times slower.

Is a consistent result with the idea that a simple loop written in a compiled langage can run with less cost than equivalent loops on interpretated languages. C is the fastest, Java is the midpoint and Python the largest tested size out of this three, but, this is not universal, there's a lot of variants to take in consideration like compiler options, memory layout, hardware and the own implementation can change the outcome.

6.3. Variation and limitations

Some measurements have a noticeably variation between repetitions, Java can be affected by just-in-time compilations and runtime warm-up, while all programs can be affected by other activities on the computers. One warm-up were used along with five measured runs were used, so a larger number of repetitions would make conclusions more reliable. The metadata environment of C is incomplete, and the three programs were not launched at the same time.

The benchmark test was made with relatively small matrices, like 150x150. The results should not be extrapolated automatically to very large matrices or Big Data workloads, normally the large-scale matrix operations uses numerical libraries, parallel processing or specialized hardware, which are outside the scope of this experiment.

7. MEMORY ESTIMATE.

n	Elements in one matrix	Estimated payload for 3 matrices
10	100	2.34 KiB
30	900	21.09 KiB
60	3600	84.38 KiB
100	10000	234.38 KiB
150	22500	527.34 KiB

The estimated element storage increases quadratically, following $O(n^2)$, while the multiplication time follows $O(n^3)$. For n=150, all three matrices have 67500 elements in total and the estimated raw payload is 540000 bytes or 527 KiB. This value will only count for the assumed storage of matrix elements; it does not represent peak RAM usage.

8. CONCLUSION.

This experiment shows that the same basic matrix multiplication algorithm can have different practical execution times depending on the programming language an its implementation. All runs recorded passed correctly all checks and the execution time increased as expected with the matrix size. In the supplied measurements, C was fastest, Java followed and Python was last, especially with larger matrices.

The conclusión is that theoretical complexity describes how work grows with input size, but it does not determine the exact time by itself, the language, runtime, compiler, data representation and measurement conditions also matter. To strengthen the experiment, the next step would be to record the exact C compiler and CPU model, repeat the tests more times, and use a more controlled environment.